TEST PLAN

IEEE 829 Standard Format

Project: Smart Expense & Budget Management Application

Version: 1.1

Date: 02 October 2026

Status: Final

Group Number: G-15

Authors:
Rhea Sushanth – PES1UG24CS371
Sarthki Vachhani – PES1UG24CS422
Sai Manogna BV – PES1UG24CS399

This plan (IEEE 829 structure, following the course Test Plan template) defines how every functional, non-functional and security requirement in the G-15 SRS will be verified, with 37 traceable test cases.

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Group G-15	First complete test plan with 37 traceable test cases
1.1	02-10-2026	Group G-15	Aligned with SRS v1.2 and Architecture & Design Specification v1.1: stale 13.2 note removed; FR-022 unit corrected; IDOR tests expect 404 and lockout 429; steps added for the password policy, category type, identical manual entries, in-file CSV duplicates, budget edit/delete and report, recurring edit and disabled category, category re-enable and admin-change logging; category seeding and TC-AVAIL-01 timing scheduled.

1. Introduction

Purpose: This document is the master test plan for the G-15 project, Smart Expense & Budget Management Application. It defines the objectives, scope, strategy, resources, schedule and exit criteria for testing, so that every functional, non-functional and security requirement in the SRS (Deliverable 2) is verified by at least one traceable test case before release. It follows the IEEE 829 test plan outline and is derived directly from the G-15 SRS (Sections 4, 5 and 8).

Scope: Testing covers the single-user web application: authentication, transactions and CSV import, budgets and alerts, reports and dashboard, recurring transactions, admin category management, and the optional sandbox bank-link adapter. Real bank linking and multi-user budgets are out of scope, as in SRS Section 1.2. No earlier version of this system exists, so there is no prior defect history to summarize.

References: G-15 Software Requirements Specification v1.2 (Deliverable 2) and Synopsis (Deliverable 1); IEEE Std 829-2008, Standard for Software and System Test Documentation; Software Architecture & Design Specification v1.1, G-15 (companion document); OWASP Top 10 (2021) and RFC 4180 (CSV format).

Definitions and abbreviations:

Term	Meaning
FR / NFR / SEC	Functional, non-functional and security requirement IDs in the SRS
TC	Test case
RTM	Requirements traceability matrix
UAT	User acceptance testing
SUT	System under test

2. Test Items

The test items are the Django apps listed in SRS Appendix E.2, tested as a single deployed build.

Item	Module	Requirements covered
Accounts app	`accounts/`	FR-001 to FR-004, SEC-001, SEC-002, SEC-004
Transactions app	`transactions/` (incl. `importer.py`, `sandbox_adapter.py`)	FR-005 to FR-011, FR-023
Budgets app	`budgets/` (incl. `alerts.py`)	FR-012 to FR-016
Reports app	`reports/`	FR-017, FR-018, NFR-001
Recurring app	`recurring/` (incl. `tasks.py`)	FR-019, FR-024
Category admin app	`categories/` (role-protected admin views)	FR-020 to FR-022

Build under test: the `main` branch of the team GitHub repository at the end of each sprint, as produced by the GitHub Actions pipeline.

3. Features to be Tested

All 24 functional, 7 non-functional and 6 security requirements in the SRS are in scope for testing; the table groups them by feature and gives the test priority (High = SRS Must have).

Feature	Requirement IDs	Test priority	Primary test type
User registration, login, logout	FR-001, FR-002, FR-004	High	Functional
Profile view and edit	FR-003	Medium	Functional
Expense and income entry	FR-005, FR-006, FR-007	High	Functional, negative
Edit and delete transaction	FR-008	Medium	Functional
CSV import, validation, duplicate skip	FR-009, FR-010, FR-011	High	Integration, negative
Budget creation, edit, delete and overlap rule	FR-012, FR-014	High	Functional, negative
Spend tracking and alerts	FR-013, FR-015, FR-016	High	Unit, system
Reports and dashboard	FR-017, FR-018	High / Medium	System
Recurring rules and scheduled generation	FR-019, FR-024	Medium / High	Integration
Admin category management	FR-020, FR-021, FR-022	Medium / Low	Functional, negative
Sandbox bank sync (stretch)	FR-023	Low	Integration (mocked)
Performance, availability	NFR-001, NFR-002	High	Performance, operational review
Usability	NFR-003	Medium	Usability walkthrough
Reliability (idempotent re-import)	NFR-004	High	Negative, integration
Maintainability, testability	NFR-005, NFR-006	Medium	Code review, unit
External policy	NFR-007	Low	Compliance review
Security controls	SEC-001 to SEC-006	High	Security, authorization

4. Features Not to be Tested

These items are excluded because the SRS places them out of scope or because they are third-party code the team does not own.

Excluded item	Reason
Real bank account linking and live payments	Out of scope (SRS 1.2, 2.5); only a mocked sandbox adapter is tested
Multi-currency conversion, investment tracking	Out of scope (SRS 1.2)
Multi-user shared or family budgets	Out of scope; possible future enhancement
Account deletion	Not offered in this release (SRS 1.2, Appendix F)
Internals of Django, HTMX, Chart.js, Django password hasher	Reused third-party components (SRS Appendix E.2); only their configuration and use are tested
Offline use	Not supported (SRS 2.6)
Localization beyond English	Single language and currency in this iteration (SRS Appendix F)

Excluded item	Reason
Live third-party sandbox API availability	Outside team control; tested only through a mock (NFR-006)

5. Test Approach / Strategy

Testing is incremental and follows the Agile sprints in SRS Appendix E.4: each sprint's features are unit and integration tested before demo, and system, security and performance testing concentrate in Sprints 5 and 6.

Test levels and techniques

Level	Technique	Tool	Requirements
Unit	White-box; boundary values on amounts, thresholds and dates	Pytest, Django test framework	FR-013, FR-015, FR-016, FR-024, NFR-005, NFR-006
Integration	Django test client against a test database; mocked sandbox API	Django test client, Pytest fixtures	FR-009 to FR-011, FR-019, FR-023
System (black-box)	Functional flows through the UI, valid and invalid inputs	Django test client, manual scripts	FR-001 to FR-008, FR-012, FR-017, FR-018, FR-020 to FR-022
Negative	Invalid, missing and malformed input; equivalence partitioning	Pytest	FR-007, FR-010, FR-014, FR-021, NFR-004
Performance	Load test on report and dashboard endpoints	Locust	NFR-001
Usability	Task-based walkthrough with 3 first-time users	Stopwatch, screen count	NFR-003
Acceptance	Sprint demo against user-story acceptance criteria	Manual	All High priority FRs
Regression	Full automated suite on every push	GitHub Actions	All automated cases

5.1 Security Validation

Security validation confirms that the controls in SRS Section 5.1.2 hold up against the OWASP Top 10 (2021) categories relevant to a Django web application. These objectives are stated in SRS Section 5.1.1 and are validated here.

Objective	Meaning for this application
Confidentiality	A user can see only their own financial data, and stored passwords are unreadable
Integrity	Only the owner can change a transaction, budget or rule, and uploaded input cannot alter the system
Accountability and access control	Admin functions are restricted, sessions expire, and security events are logged

SEC ID	Objective	Validation method	OWASP category	Test case
SEC-001	Confidentiality	Inspect stored `password_hash` values; confirm a salted PBKDF2/Argon2 hash and no plaintext in DB or logs	A02 Cryptographic Failures	TC-SEC-01

SEC ID	Objective	Validation method	OWASP category	Test case
SEC-002	Access control	Log in as Customer and request admin category URLs directly; expect HTTP 403	A01 Broken Access Control	TC-SEC-02
SEC-003	Integrity	Upload CSV with script tags, formula injection (`=cmd|...`), oversized and non-UTF-8 content; submit SQL and HTML payloads in note fields	A03 Injection	TC-SEC-03
SEC-004	Access control	Call authenticated endpoints with no session, an expired session and a logged-out session; expect redirect or 401	A07 Identification and Authentication Failures	TC-SEC-04
SEC-005	Confidentiality	Review models, migrations and logs for stored bank tokens	A04 Insecure Design	TC-SEC-05
SEC-006	Accountability	Trigger failed logins, a forbidden request and an admin category change; confirm log entries with timestamp, user and IP	A09 Logging and Monitoring Failures	TC-SEC-06
Extra	Confidentiality	User A requests user B's transaction and budget IDs in the URL (IDOR); expect 404	A01 Broken Access Control	TC-SEC-07

The planned penetration testing from SRS Appendix D is a manual OWASP-style pass in Sprint 5 using the browser and OWASP ZAP baseline scan against the staging build. Findings rated High or Critical block release.

6. Test Environment

Item	Setting
Server	Python 3 and Django, run under the test server locally and on CI
Database	SQLite for unit and integration runs; PostgreSQL on staging for performance and system runs
Browsers	Latest Chrome, Edge, Firefox, plus one mobile browser
Test data	Seed script creating 3 users (2 Customers, 1 Administrator), 8 categories, and 10,000 transactions for load tests; sample CSV files (valid, malformed, duplicate, oversized)
External services	Sandbox bank API replaced by a mock adapter (NFR-006)
Tools	Pytest, Django test client, Coverage.py, Locust, OWASP ZAP, Postman
CI	GitHub Actions runs the full suite on every push and pull request

7. Test Schedule

Sprint (weeks)	Testing focus
1 (0-2)	Login unit tests, CI pipeline running
2 (2-4)	Transaction entry, category rules and CSV import (transactions module), unit tests
3 (4-6)	Budget calculation and overlap rule, unit and integration tests
4 (6-8)	Report and dashboard system tests, first performance run
5 (8-10)	Alerts, recurring, admin tests; security validation (5.1); hardening
6 (10-12)	Full regression, performance re-run, usability walkthrough, 7-day availability polling on staging from the start of week 10 (TC-AVAIL-01), final report

Milestones follow the SRS Appendix E.4 Gantt chart: test case design completes alongside each sprint's feature freeze, and the final regression and report are produced in the last two days of Sprint 6. Until the admin panel is built in Sprint 5, transaction and budget tests use the 8 categories created by the seed script (Section 6).

8. Test Deliverables

Deliverable	Format	Owner
This test plan and RTM	Doc	Group G-15
Test cases and test scripts	Pytest files in `tests/` per app	Feature owner
CI pipeline run log	GitHub Actions	Rhea Sushanth
Coverage report	Coverage.py HTML and XML	Rhea Sushanth
Performance test report	Locust CSV and summary	Rhea Sushanth
Security test findings	Issue list and ZAP baseline report	Sarthki Vachhani
Usability walkthrough notes	Timing sheet	Sarthki Vachhani
Defect log	GitHub Issues	All
Final test summary report	Doc	All

9. Roles and Responsibilities

Each member writes and runs tests for the feature they own (as in the Synopsis); shared and cross-cutting testing is split as shown.

Area	Owner
Authentication, profile, RBAC, security tests (TC-AUTH, TC-SEC)	Sarthki Vachhani
Transactions, CSV import, sandbox adapter (TC-TXN)	Sai Manogna BV
Budgets, alerts, reports, recurring, performance, CI (TC-BUD, TC-RPT, TC-REC, TC-PERF)	Rhea Sushanth
Admin category tests (TC-ADM) and usability walkthrough	Sarthki Vachhani
Regression run and final test summary	All

10. Risks and Mitigation

Risk	Impact	Mitigation
Sandbox bank API unavailable	FR-023 cannot be tested live	Test only against the mock adapter; drop the live test if no sandbox exists
Scheduler timing is hard to test	FR-019 and FR-024 flaky	Inject a fixed clock into the rule runner and call it directly
Test load environment differs from production	NFR-001 result not representative	Run the load test on staging PostgreSQL with the seeded 10,000 transactions
Short sprints leave little time for security testing	Late defects	Write SEC tests in Sprint 3, run them on every push, and hold the ZAP pass in Sprint 5
Only three members	Testing delays	Each member automates tests for their own feature as they build it

Risk	Impact	Mitigation
CI unavailable or rate limited	No regression signal	Tests also run locally with one Pytest command

11. Assumptions & Dependencies

- The system stays single-user and single-currency for this release, as scoped in SRS Section 1.2; tests are not written against multi-user or multi-currency behavior.
- The test environment (Python/Django version, SQLite/PostgreSQL, browsers) matches the tool stack fixed in SRS Appendix E.5 and will not change mid-project.
- No live sandbox bank credentials will be issued for grading, so FR-023 and NFR-006 are verified against the mock adapter only; a real sandbox, if made available, would be tested as a bonus pass.
- The three-person team and the ownership split in the Synopsis remain stable through Sprint 6; the schedule in Section 7 assumes no member drops out.
- GitHub Actions remains free and available for the life of the project so the CI-dependent deliverables (pipeline log, regression gate) can be produced.
- Test data is synthetic only, consistent with NFR-007 and the SRS constraint against using real personal financial data.

12. Suspension & Resumption Criteria

- Suspend testing of a module if the build fails to start, the database migration fails, or a Critical defect blocks more than 30% of that module's cases.
- Resume when a fixed build passes the smoke test (TC-AUTH-01 and TC-AUTH-02) on CI, and the failed cases are re-run.

13. Test Case Management & Traceability

Test cases are managed as Pytest files per app (Section 8) and as the tables below, which give the exact steps, input and expected result for each of the 37 cases so they can be automated directly. Twenty-three cases cover functional requirements (13.1) and fourteen cover non-functional and security requirements (13.2); 13.3 is the full requirements traceability matrix.

13.1 Functional test cases

Twenty-three test cases cover FR-001 to FR-024; every case uses the ID already seeded in the SRS traceability matrix (Section 8) where one exists.

Accounts (owner: Sarthki Vachhani)

TC ID	Req	Test steps and input	Expected result	Type
TC-AUTH-01	FR-001	1. Open Register. 2. Enter name Asha, email `asha@test.com`, password `Str0ng#Pass1`. 3. Submit. Then repeat with the same email. Then register `bob@test.com` with password `abc123` (6 characters) and with password `abcdefgh` (no digit).	First submit creates the account and redirects to the dashboard. Second submit is rejected with the error "Email already registered"; only one user row exists. Both weak passwords are rejected with the password-policy error and no account is created.	Functional, negative
TC-AUTH-02	FR-002	1. Log in with `asha@test.com` and the correct password. 2. Log out. 3. Log in with a wrong password.	Correct login opens an authenticated session. Wrong password shows a generic error and creates no session.	Functional
TC-AUTH-03	FR-003	Logged in: open Profile, change name to Asha K and save; then submit an empty name.	Name updates and persists after reload. Empty name is rejected with an inline error.	Functional
TC-AUTH-04	FR-004	Log in, note the session cookie, click Logout, then press the browser Back button and re-request the dashboard URL.	Session is invalidated; the dashboard request redirects to Login. The old cookie is rejected.	Functional

Transactions and CSV import (owner: Sai Manogna BV)

TC ID	Req	Test steps and input	Expected result	Type
TC-TXN-01	FR-005	Add expense: amount 250.00, category Food, date today, note "Lunch". Then add the same expense again.	Transaction saved with type Expense and source Manual; it appears in the list and the note is optional. The second identical manual expense is also saved (duplicate skipping applies to CSV import only).	Functional
TC-TXN-02	FR-006	Add income: amount 20000.00, category Salary, date today, no note.	Transaction saved with type Income and source Manual.	Functional
TC-TXN-03	FR-007	Submit a transaction with no category; then with amount 0, with amount -5, and an Income transaction with category Food (an Expense category).	Each submission is rejected with an inline error and nothing is saved. A submission with exactly one active category of the matching type saves.	Negative, positive
TC-TXN-04	FR-008	Edit own transaction amount from 250 to 300, then delete it. As user B, request user A's transaction edit URL.	Edit and delete succeed for the owner. User B receives 404 and the record is unchanged.	Functional
TC-TXN-05	FR-009, FR-010	Import a 10-row CSV (columns date,amount,type,category,note) with 7 valid rows and 3 malformed rows (bad date, negative amount, missing category).	7 rows imported with source CSV. The 3 malformed rows are not imported and are listed with row number and reason.	Integration, negative
TC-TXN-06	FR-023	With the mock sandbox adapter returning 5 transactions, click Sync. Repeat with the adapter set to fail.	5 transactions imported with source Sandbox. On failure the user sees a clear message and no partial data is saved.	Integration (mocked)

TC ID	Req	Test steps and input	Expected result	Type
TC-TXN-07	FR-011	Import a CSV containing a row whose date, amount and note match an existing transaction, 2 new rows, and a fourth row that repeats one of the 2 new rows.	The existing-match row and the repeated in-file row are both skipped and reported as duplicates; the 2 new rows are imported once each and the import is not rolled back.	Integration

Budgets and alerts (owner: Rhea Sushanth)

TC ID	Req	Test steps and input	Expected result	Type
TC-BUD-01	FR-012	Create a monthly Food budget with limit 5000 and default threshold. Then create a weekly Travel budget. Edit the Travel limit to 2000, then delete the Travel budget. Try to create a budget on the Salary (Income) category.	Both budgets are saved with threshold 80 and the chosen period. Limit 0, a threshold of 101 and the budget on an Income category are rejected. The edit is saved; after delete the Travel budget and its alerts no longer appear.	Functional
TC-BUD-02	FR-013	With a Food budget of 5000, add expenses of 1000 and 1500, and delete the 1000 one.	Displayed spend is 1000, then 2500, then 1500. Each figure appears within 2 seconds of the save. Income is not counted.	Unit, system
TC-BUD-03	FR-014	With an active monthly Food budget for this month, create another monthly Food budget for the same month. Then create a monthly Food budget for next month.	The second budget for the same category and month is rejected with an overlap error. The budget for next month is saved.	Negative
TC-BUD-04	FR-015	Food budget 5000, threshold 80. Add spend to 3999, then to 4000, then add another 100.	No alert at 3999 (79.98%). Exactly one alert at 4000 (80%). No second threshold alert at 4100.	Functional, boundary
TC-BUD-05	FR-016	Continue the same budget: add spend to 5000, then to 5001.	No over-budget alert at exactly 5000. One over-budget alert at 5001 (above 100%), with `is_read` false.	Functional, boundary

Reports, dashboard and recurring transactions (owner: Rhea Sushanth)

TC ID	Req	Test steps and input	Expected result	Type
TC-RPT-01	FR-017	Seed Food 1000, Rent 3000, Travel 1000 for one month. Open the category report for that month, then for an empty month. With a Food budget of 5000 for that month, open the budget report.	Report shows Food 20%, Rent 60%, Travel 20% and a total of 5000. Empty month shows totals of 0 and no error. The budget report shows Food with limit 5000, spent 1000 and remaining 4000.	System
TC-RPT-02	FR-018	Open the dashboard with seeded data.	Pie, bar and line charts render from the same data as the report; the browser console shows no errors.	System
TC-REC-01	FR-019	Create a monthly rule: amount 999, category Subscriptions, next run date tomorrow. Edit the amount to 1099. Then deactivate the rule.	Rule is saved and active, and the edit is saved. A disabled category or amount of 0 is rejected on create and on edit. After deactivation the rule shows as inactive.	Integration

TC ID	Req	Test steps and input	Expected result	Type
TC-REC-02	FR-024	Run the rule runner with the clock set to (a) the day before, (b) the run date, (c) after the rule is deactivated, (d) the run date of a rule whose category has been disabled, with a second active rule also due.	(a) no transaction. (b) exactly one transaction with source Recurring, and `next_run_date` moves forward one month. (c) no transaction. Running it twice on the same day creates only one. (d) no transaction for that rule and the rule is flagged on the recurring page; the second rule's transaction is still created.	Integration

Administrator (owner: Sarthki Vachhani)

TC ID	Req	Test steps and input	Expected result	Type
TC-ADM-01	FR-020	As admin, add category "Gym" (Expense), rename it to "Fitness", disable it, then re-enable it.	Each change is saved; after disabling `is_active` is false, and after re-enabling `is_active` is true.	Functional
TC-ADM-02	FR-021	With "Fitness" disabled, try to use it in a new transaction, budget and recurring rule as a Customer. Then re-enable it and repeat.	All three attempts are rejected while disabled. Existing transactions that used it remain visible. After re-enabling, all three are accepted.	Negative
TC-ADM-03	FR-022	As admin, open the user list; then request it as a Customer.	Admin sees the list of registered users with no password data. The Customer request returns 403.	Functional

13.2 Non-functional and security test cases

Fourteen further cases cover NFR-001 to NFR-007 and SEC-001 to SEC-006. The test load (NFR-001) and operating window (NFR-002) used below are the values fixed in SRS Section 5.

Non-functional requirements

TC ID	Req	Test steps and input	Expected result	Type
TC-PERF-01	NFR-001	Seed 10,000 transactions. Run Locust with 50 concurrent users for 10 minutes, each alternating dashboard, category report and budget page requests.	95th percentile response time is 2.0 seconds or less and the error rate is below 1%.	Performance
TC-AVAIL-01	NFR-002	Poll the staging health URL every 5 minutes for 7 days during the operating window 06:00-24:00 IST, excluding announced maintenance.	At least 99% of polls return HTTP 200. Downtime causes are logged.	Operational review
TC-USA-01	NFR-003	Three first-time users, given only the goal "record a Rs. 200 food expense", start from the logged-in dashboard while an observer counts screens.	Every user completes the task in 3 primary screens or fewer, and without help.	Usability walkthrough
TC-REL-01	NFR-004	Import file A (20 rows) three times. Then interrupt an import after 10 rows and import file A again.	Transaction count is 20 after every run, with no duplicate rows and no half-imported batch.	Negative, integration

TC ID	Req	Test steps and input	Expected result	Type
TC-MNT-01	NFR-005	Review the repository layout and run `coverage run -m pytest` on the budgets, recurring and transactions apps.	Budgeting, alerts and recurrence logic sit in separate apps, each with unit tests, and line coverage is 70% or more.	Code review, unit
TC-TST-01	NFR-006	Disable network access and unset the sandbox adapter, then run the budget calculation and alert evaluation test modules.	All tests pass without any external bank or payment service.	Unit, integration
TC-POL-01	NFR-007	Review the deployment against a four-point checklist: hosted on campus-approved infrastructure, no real personal financial data, test data only, access limited to the team and evaluators.	All four checklist items are confirmed and the reviewer signs the checklist.	Compliance review

Security requirements (validation approach in Section 5.1)

TC ID	Req	Test steps and input	Expected result	Type
TC-SEC-01	SEC-001	Register a user with password `Str0ng#Pass1`. Query the user table and search logs and database dump for the password.	Stored value is a salted hash (for example `pbkdf2_sha256$...`); the plaintext appears nowhere.	Configuration review
TC-SEC-02	SEC-002	Log in as a Customer and request the admin category and user-list URLs directly, by GET and POST.	Every request returns 403 and no category is changed.	Authorization
TC-SEC-03	SEC-003	Upload CSVs with a `<script>` note, a formula cell `=HYPERLINK(...)`, a 10 MB file and a non-UTF-8 file. Enter `' OR 1=1 --` in note and name fields.	Content is escaped on display, formulas are stored as text, oversized and wrongly encoded files are rejected, and the SQL string is stored literally.	Negative, security
TC-SEC-04	SEC-004	Call the transaction list and budget endpoints with no session cookie, a tampered cookie, a session idle for more than 30 minutes, and a logged-out session.	Each request is redirected to Login (or returns 401) and returns no data.	Security
TC-SEC-05	SEC-005	Inspect models, migrations and logs while running a mocked sandbox sync.	No bank access token or credential is stored in the database or written to logs.	Design review
TC-SEC-06	SEC-006	Make 5 failed logins within 10 minutes, then a 6th attempt with the correct password, and one forbidden admin request; as admin, rename a category; then read the security and audit logs.	The 6th attempt is refused with HTTP 429 because the account is locked for 10 minutes. The category rename appears in the audit log. Each event is logged with timestamp, user or email, IP address and event type; no password is logged.	System
TC-SEC-07	SEC-002, Confidentiality	As user A, request user B's transaction, budget and alert IDs by changing the ID in the URL.	Every request returns 404 and B's data is not exposed.	Authorization (IDOR)

13.3 Requirements traceability matrix

Every one of the 37 SRS requirements maps to at least one test case, and every test case maps back to at least one requirement. The table extends the SRS Section 8 matrix (forward: requirement to use case, implementation unit and test; backward: the Req column in Section 13.1 and 13.2).

Req ID	Use case	Implementation unit	Test case(s)
FR-001	UC-01	`accounts/views.py`	TC-AUTH-01
FR-002	UC-01	`accounts/views.py`	TC-AUTH-02
FR-003	UC-01	`accounts/views.py`	TC-AUTH-03
FR-004	UC-01	`accounts/views.py`	TC-AUTH-04
FR-005	UC-02	`transactions/models.py`, `views.py`	TC-TXN-01
FR-006	UC-02	`transactions/models.py`, `views.py`	TC-TXN-02
FR-007	UC-02	`transactions/forms.py`	TC-TXN-03
FR-008	UC-02	`transactions/views.py`	TC-TXN-04
FR-009	UC-05	`transactions/importer.py`	TC-TXN-05
FR-010	UC-05	`transactions/importer.py`	TC-TXN-05
FR-011	UC-05	`transactions/importer.py`	TC-TXN-07
FR-012	UC-03	`budgets/models.py`	TC-BUD-01
FR-013	UC-03	`budgets/services.py`	TC-BUD-02
FR-014	UC-03	`budgets/models.py` (validation)	TC-BUD-03
FR-015	UC-07	`budgets/alerts.py`	TC-BUD-04
FR-016	UC-07	`budgets/alerts.py`	TC-BUD-05
FR-017	UC-04	`reports/views.py`	TC-RPT-01
FR-018	UC-04	`reports/views.py`	TC-RPT-02
FR-019	UC-08	`recurring/tasks.py` (rule runner), `recurring/models.py`	TC-REC-01
FR-020	UC-06	`categories/admin.py`	TC-ADM-01
FR-021	UC-06	`categories/models.py`	TC-ADM-02
FR-022	UC-09	`categories/admin.py`	TC-ADM-03
FR-023	UC-10	`transactions/sandbox_adapter.py`	TC-TXN-06
FR-024	UC-11	`recurring/tasks.py`	TC-REC-02
NFR-001	UC-04	`reports/queries.py`	TC-PERF-01
NFR-002	All	Deployment	TC-AVAIL-01
NFR-003	UC-02	Transaction entry templates	TC-USA-01
NFR-004	UC-05	`transactions/importer.py`	TC-REL-01, TC-TXN-07
NFR-005	All	Project structure	TC-MNT-01
NFR-006	UC-03, UC-07	`budgets/services.py`, `alerts.py`	TC-TST-01
NFR-007	All	Deployment	TC-POL-01
SEC-001	UC-01	`accounts/models.py`	TC-SEC-01
SEC-002	UC-06, UC-09	`accounts/permissions.py`	TC-SEC-02, TC-SEC-07, TC-ADM-03
SEC-003	UC-05	`transactions/importer.py`, forms	TC-SEC-03
SEC-004	UC-01	Auth middleware	TC-SEC-04, TC-AUTH-04
SEC-005	UC-10	`transactions/sandbox_adapter.py`	TC-SEC-05

Req ID	Use case	Implementation unit	Test case(s)
SEC-006	UC-01	Security logging in `accounts/`	TC-SEC-06

Coverage summary: 24 of 24 FRs, 7 of 7 NFRs and 6 of 6 SECs have a test case; 37 test cases in total (23 functional, 7 non-functional, 7 security).

14. Test Metrics & Reporting

Exit criteria:

- Sprint exit: 100% of High-priority test cases for that sprint's features pass, and no open Critical or High defects.
- Release exit (Sprint 6): at least 95% of all test cases pass, all High-priority requirements pass, and every failed Medium or Low case has a documented workaround.
- Coverage: at least 70% line coverage on the budgets, recurring and transactions business-logic modules (supports NFR-005).
- Performance: NFR-001 passes only if the 95th percentile response is 2 seconds or less at the load stated in TC-PERF-01.
- Security: zero open High or Critical findings from Section 5.1.

Metrics collected every sprint:

- % of planned test cases executed
- % passed / failed, split by functional, non-functional and security
- Defect density per app (accounts, transactions, budgets, reports, recurring, categories)
- Defect aging (days open) for Critical and High defects
- Requirement coverage (FRs, NFRs, SECs with at least one passing test case)
- Line coverage percentage from Coverage.py

Reporting: a short test summary is posted to the team channel after each sprint's regression run, and the final test summary report (Section 8) consolidates all of the above against the exit criteria for the Sprint 6 release decision. A test case passes when the actual result matches the expected result exactly; otherwise it fails and a defect is logged as a GitHub issue.

15. Approvals

Role	Name	Signature / Date
Prepared by (Budgets, Reports and CI Pipeline)	Rhea Sushanth	Rhea, 02-10-2026
Prepared by (Auth, Security and Admin testing)	Sarthki Vachhani	Sarthki, 02-10-2026
Prepared by (Transactions and Import testing)	Sai Manogna BV	Manogna, 02-10-2026